
SYMMETRY-RESTRICTED SAT ENCODINGS, LEAN 4 FORMALIZATION, AND VERIFICATION LIMITS FOR CONWAY’S 99-GRAPH

TECHNICAL NOTE / BENCHMARK REPORT

Antonio Machuca

Department of Computer Science and Statistics
Universidad Rey Juan Carlos
Madrid, Spain

`am.machuca.2023@alumnos.urjc.es`

Permanent contact: `contactoantoniomachuca@gmail.com`

September 2026

ABSTRACT

Conway’s 99-graph problem (1970) asks for the existence of a strongly regular graph with parameters $(99, 14, 1, 2)$. Theoretical results restrict the prime divisors of the automorphism group order to $\{2, 3\}$. However, automated searches face large combinatorial search spaces. This paper presents a verified benchmark framework that combines interactive theorem proving in Lean 4 with certified propositional satisfiability (SAT) solving. We introduce a four-state taxonomy—PROVED, COMPILED, EXPLORED, and PENDING—to separate kernel-verified theorems, certified clausal refutations, tested compilers, and open conjectures. In Lean 4, we kernel-verify core structural invariants: K_4 -freeness, 1-factor neighborhood decomposition $7K_2$, diameter at most 2, and decidability of the adjacency matrix equation. These proofs depend solely on the foundational axioms `[propext, Quot.sound]`. We also formalize the arithmetic contradictions that underlie character-theoretic obstructions. This distinguishes verified arithmetic deductions from unverified geometric action hypotheses. For propositional logic, we implement canonical SAT encodings with Crawford symmetry-breaking predicates. A 39-test regression suite verifies the compiler pipeline after eliminating literal/constant collision defects. Automorphisms of order 7 were excluded computationally by Behbahani and Lam (2011) and reduced computer-free to $\mathbb{Z}_7$ by Cesarz and Woldar (2025). However, Thakkar (2026) reported an open timeout on this case using CP-SAT. We provide an independently verifiable clausal refutation. CaDiCaL refutes the canonical encoding in 771.59 s and produces a 192.2 MB (183.31 MiB) binary proof. The proof checker `drat-trim` verifies this proof in 844.01 s. We also analyze the incidence geometry of the open involution case ($f = 1$, governed by $CC^T = 10I + 2J$) and fixed-point-free order-3 actions. A multi-threaded solver campaign ran for 12.5 days and accumulated over 2.4 billion conflicts without finding a satisfying assignment. All Lean modules, SAT compilers, test suites, and proof certificates are fully reproducible.

Keywords Conway 99-graph · strongly regular graphs · interactive theorem proving · Lean 4 · SAT solving · DRAT certification · symmetry breaking

1 Introduction and Literature Attribution

A strongly regular graph $\text{srg}(v, k, \lambda, \mu)$ is a regular graph on v vertices with degree k . Every pair of adjacent vertices has exactly λ common neighbors. Every pair of distinct non-adjacent vertices has exactly μ common neighbors. In 1970, John H. Conway asked whether a strongly regular graph exists with parameters $(99, 14, 1, 2)$ [1]. He offered a \$1,000 prize for its resolution. Over five decades later, the existence of this graph remains an open problem in algebraic combinatorics [2, 3].

Let $A \in \{0, 1\}^{99 \times 99}$ denote the adjacency matrix of a Conway 99-graph. The strongly regular parameters require A to satisfy the matrix equation

$$A = A^T, \quad \text{diag}(A) = 0, \quad A^2 + A = 12I_{99} + 2J_{99}, \quad (1.1)$$

where I_n is the $n \times n$ identity matrix and J_n is the $n \times n$ all-ones matrix. Spectral graph theory gives the eigenvalues of A : $k = 14$ (multiplicity 1), $r = 3$ (multiplicity 54), and $s = -4$ (multiplicity 44):

$$\text{Spec}(A) = \{14^1, 3^{54}, (-4)^{44}\}. \quad (1.2)$$

For each vertex $x \in V(G)$, let $\Gamma(x)$ denote its open neighborhood. Let $\Gamma_2(x) = V(G) \setminus (\{x\} \cup \Gamma(x))$ denote the second subconstituent. Because $k = 14$ and $\lambda = 1$, each vertex in $\Gamma(x)$ has exactly one neighbor in $\Gamma(x)$. The induced subgraph $G[\Gamma(x)]$ is therefore a 1-regular graph of seven disjoint edges ($7K_2$). The second subconstituent $\Gamma_2(x)$ contains $99 - 1 - 14 = 84$ vertices. Because $\mu = 2$, each vertex $y \in \Gamma_2(x)$ is adjacent to exactly two vertices in $\Gamma(x)$ [2, 3].

1.1 Literature Attribution of Automorphism Group Bounds

An unconstrained combinatorial search on a 99-vertex graph involves $2^{\binom{99}{2}} = 2^{4851}$ possible edge configurations. Computational searches must therefore assume automorphisms. Prior literature establishes strict bounds on the automorphism group $\text{Aut}(G)$:

1. **Involutions (order 2) fix exactly one vertex.** Makhnev and Minakova (2001, 2004) [4] classified prime-order automorphisms of strongly regular graphs with $\lambda = 1$ and $\mu = 2$. Makhnev (2009) [5] summarized that an involution $t \in \text{Aut}(G)$ cannot fix 3, 5, 7, or more vertices due to character integrality. Any involution of a Conway 99-graph must therefore fix exactly $f = |\text{Fix}(t)| = 1$ vertex.
2. **Automorphisms of order 3 are fixed-point-free; orders 6 and 9 are excluded.** Behbahani and Lam (2011) [6] proved that an automorphism of order 3 on $\text{srg}(99, 14, 1, 2)$ has no fixed points. It acts as a fixed-point-free (FPF) permutation with 33 cycles of length 3. Crnković and Maksimović (2014) [7] proved that $\text{Aut}(G)$ contains no subgroups of order 6 or 9. This result excludes $\mathbb{Z}_6$, S_3 , $\mathbb{Z}_9$, and $\mathbb{Z}_3 \times \mathbb{Z}_3$.
3. **Prime orders $p \geq 5$ are excluded.** Behbahani and Lam (2011) [6] showed that the only possible prime divisors of $|\text{Aut}(G)|$ are 2 and 3. They excluded automorphisms of order 5, 7, and higher primes through computer-assisted orbit matrix enumeration. Cesarz and Woldar (2025) [3] gave computer-free proofs that:

$$2 \mid |\text{Aut}(G)| \implies |\text{Aut}(G)| \mid 6, \quad 7 \mid |\text{Aut}(G)| \implies \text{Aut}(G) \cong \mathbb{Z}_7. \quad (1.3)$$

Combining this even-order implication with Crnković and Maksimović’s exclusion of order 6 proves that if $2 \mid |\text{Aut}(G)|$, then $|\text{Aut}(G)| = 2$. Therefore, $\text{Aut}(G)$ contains no elements of order 4 or 6.

4. **Computational benchmark context.** Thakkar (2026) [8] introduced forced-structure reductions and encoded symmetry classes using Google OR-Tools CP-SAT. The solver timed out on the single-fixed-point order-7 action, returning UNKNOWN after 48 hours on 14 cores. This left open the task of producing an independently verified computational certificate for that case.

1.2 Contributions and Scope of this Benchmark Report

This work provides a verified computational benchmark that combines interactive theorem proving in Lean 4 with certified SAT solving. The primary contributions are:

- We introduce a four-state taxonomy (PROVED, COMPILED, EXPLORED, PENDING) to clarify proof verification and modeling limits.
- In Lean 4, we verify structural invariants (K_4 -freeness, 1-factor neighborhood decomposition, diameter at most 2) and the decidability of the adjacency conditions. The Lean kernel checks these proofs using only `[propext, Quot.sound]`. We also formalize the arithmetic contradictions that arise in character-theoretic analyses.
- We implement canonical SAT compilers with Crawford symmetry-breaking predicates [9]. A 39-test regression suite verifies the compiler pipeline across syntax, coordinate models, and truth tables.
- We resolve the computational benchmark for order 7. CaDiCaL refutes the canonical encoding in 771.59 s and generates a 192.2 MB (183.31 MiB) binary proof. The checker `drat-trim` independently verifies the proof in 844.01 s.
- We determine the incidence geometry of the open involution case $f = 1$ ($CC^T = 10I + 2J$). We report on a multi-threaded cloud campaign that accumulated over 2.4 billion conflicts across Z_2 ($f = 1$) and Z_3 (FPF) without finding a satisfying assignment.

2 Lean 4 Formalization and Operational Taxonomy

To prevent ambiguity between mechanized proofs, empirical solver outputs, and open conjectures, we classify every result and artifact into one of four operational states:

PROVED. A theorem verified by the Lean 4 kernel using only the foundational axioms `[propext, Quot.sound]`; or a propositional formula whose refutation is certified by an independent proof checker (`drat-trim`).

COMPILED. Formally typed Lean 4 code compiling with `sorryAx` stubs for open subgoals; or Python SAT compilers whose clausal outputs pass deterministic regression tests.

EXPLORED. Combinatorial searches, matrix calculations, and multi-threaded solver executions that finished without finding a satisfying assignment or producing a proof certificate.

PENDING. Open mathematical questions (such as graph existence or full rigidity $|\text{Aut}(G)| = 1$) and unfinished formalizations connecting group actions to arithmetic hypotheses.

2.1 Kernel-Verified Structural Properties in Lean 4

We formalize the adjacency definition of a Conway 99-graph in Lean 4 [10] with the predicate `ConwayAdj`. This predicate enforces symmetry, zero diagonal, binary entries, and the matrix equation (1.1). To support verified computation, we define a computable Boolean function `checkConway` and prove its equivalence to `ConwayAdj`:

$$\text{conway_soundness, conway_complete} : \quad \forall A \in \mathbb{N}^{99 \times 99}, \quad \text{checkConway}(A) = \text{true} \iff \text{ConwayAdj}(A). \quad (2.1)$$

In the modules `Structural.lean` and `Z2Classification.lean`, we prove structural invariants directly from `ConwayAdj`:

- `conway_k4_free` (`Structural.lean`): Every graph satisfying the matrix equation contains no complete subgraph K_4 , because $\lambda = 1$.
- `conway_neighborhood_one_factor` (`Structural.lean`): For each vertex x , the open neighborhood $G[\Gamma(x)]$ is a 1-regular graph of seven disjoint edges ($7K_2$).
- `conway_diameter_at_most_two` (`Structural.lean`): Any two non-adjacent vertices have common neighbors because $\mu = 2 > 0$. Therefore, $\text{diam}(G) \leq 2$.
- `conway_z2_f3_fixed_points_dichotomy` (`Z2Classification.lean`): If an involution fixes three vertices, their induced subgraph is either a triangle K_3 or an independent set $3K_1$.

The Lean kernel checked all these declarations with `#print axioms`. They depend solely on `[propext, Quot.sound]`.

2.2 The Boundary Between Arithmetic Refutations and Graph Hypotheses

Character-theoretic analyses of automorphisms impose numerical constraints on fixed edge counts and eigenspace multiplicities. In the Lean library, we kernel-verify the arithmetic refutations of these systems:

- For an involution with $f = 5$ fixed vertices, the declaration `conway_z2_f5_spectral_formula_contradiction` refutes the existence of an integer ε_1 satisfying:

$$(\varepsilon_1 = 18 \vee \varepsilon_1 = 15) \wedge (\varepsilon_1 \bmod 7 = 6). \quad (2.2)$$

- For $f = 7$, `conway_no_z2_f7_formula_automorphism` refutes:

$$\varepsilon_1 \in \{7, 10, 13\} \wedge (\varepsilon_1 \bmod 7 = 2). \quad (2.3)$$

- In `CesarzWoldarTheorems.lean`, terminal arithmetic contradictions such as $7a = 62$ or sums of even integers equaling 5 are verified using only `[propext, Quot.sound]`.

These arithmetic refutations are **PROVED**. However, their wrappers (such as `conway_no_z2_f5_automorphism`) take the counting identities and spectral congruences as hypotheses. Deducing these arithmetic hypotheses directly from `ConwayAdj(A)` and an involution t in Lean remains **PENDING**. Likewise, the classification theorem `conway_automorphism_group_restricted` uses `sorryAx` for open subcases, placing its status at **COMPILED**.

3 Canonical SAT Encodings and Symmetry Breaking

To explore automorphism classes computationally, we map the graph constraints under a cyclic automorphism group $\langle g \rangle \cong \mathbb{Z}_p$ into Conjunctive Normal Form (CNF).

3.1 Orbit Parameterization and Crawford Symmetry Cuts

Under the action of an automorphism g of prime order p , the 99 vertices partition into m orbits $\mathcal{O}_1, \dots, \mathcal{O}_m$, where $|\mathcal{O}_i| \in \{1, p\}$. The adjacency between two orbits of size p is invariant under cyclic shifts. A circulant Boolean vector $c_{i,j} \in \{0, 1\}^p$ parameterizes this adjacency, where $c_{i,j,k} = 1$ if vertex $0 \in \mathcal{O}_i$ connects to vertex $k \in \mathcal{O}_j$.

The strongly regular graph conditions translate into exact cardinality constraints:

$$\sum_{u \in \Gamma(x)} A_{u,v} = \begin{cases} 14 & \text{if } x = v, \\ 1 & \text{if } x \sim v, \\ 2 & \text{if } x \not\sim v. \end{cases} \quad (3.1)$$

We convert these pseudo-Boolean constraints to CNF with Tseitin transformations, using sequential counters and sorting networks.

To prune isomorphic search spaces, we enforce symmetry-breaking predicates following Crawford et al. [9]. Let $\mathcal{G} \leq \text{Sym}(m)$ be the automorphism group preserving the orbit partition. For each non-identity generator $\pi \in \mathcal{G}$, we enforce a lexicographical leader constraint: the assignment vector X must precede its permuted image $\pi(X)$:

$$X \leq_{\text{lex}} \pi(X). \quad (3.2)$$

For variables $(x_1, \dots, x_n)$ and their images $(y_1, \dots, y_n) = (\pi(x_1), \dots, \pi(x_n))$, we introduce auxiliary variables $e_i \leftrightarrow (x_i = y_i)$ and enforce:

$$(\neg x_i \wedge y_i) \vee (e_i \wedge (X_{i+1..n} \leq_{\text{lex}} Y_{i+1..n})). \quad (3.3)$$

3.2 Compiler Verification and Defect Repair

Translating graph constraints into propositional clauses requires careful validation. In prototype implementations, primary Boolean variables were 1-indexed (with ID 1 for x_1), while helper functions such as `get_and_lit` and `add_card_equals` treated integer 1 as the Boolean constant `True`. When encoding $x_1 + x_2 = 1$ via `add_card_equals([1, 2], 1)`, the compiler evaluated literal 1 as constant `True`. This produced the degenerate unit clause $[-2]$, which enforced $x_2 = 0$ while omitting x_1 . The formula then admitted $(x_1, x_2) = (0, 0)$ and rejected $(0, 1)$.

Example 3.1 (The Literal-1 Collision Defect). The collision of variable ID 1 with the Boolean constant `True` changes an exact-1 constraint on $\{x_1, x_2\}$ into the unit clause $\neg x_2$. This defect shows why compiler primitives require end-to-end truth table tests.

We refactored the compiler architecture to separate variable indices from Boolean constants. Variables are now allocated through explicit Tseitin gate managers, reserving index 0 as an internal sentinel.

To verify compiler correctness, we created a deterministic regression suite of 39 tests in `tests/`:

1. **Compiler-Helper Unit Tests (20 tests):** Verifies truth tables for conjunctions, cardinality constraints, circulant orbit variables, and Crawford lex-leader cuts for Z_7 , Z_3 fixed-3, and Z_3 FPF compilers.
2. **Geometric and Input-Audit Tests (11 tests):** Evaluates 2,394 cardinality constraints symbolically against the coordinate model, checking branch cut consistency and index mappings.
3. **Supervisor Process Tests (8 tests):** Validates bounded execution, process lifecycle management, wall-time enforcement, and proof byte limits.

All 39 tests pass deterministically in ~ 5.5 s. This confirms COMPILED status for the SAT generation pipeline.

4 Certified Refutation of Order 7

Behbahani and Lam (2011) [6] excluded automorphisms of order 7 using orbit matrices. Cesarz and Woldar (2025) [3] reduced the case $7 \mid |\text{Aut}(G)|$ to $\text{Aut}(G) \cong \mathbb{Z}_7$ without computers. However, Thakkar (2026) [8] reported that Google OR-Tools CP-SAT timed out on this case. We provide an independently verifiable proof certificate that refutes this symmetry class.

4.1 Encoding Parameters and Formula Statistics

Under a cyclic automorphism g of order 7, the 99 vertices partition into:

- Exactly 1 fixed vertex x_0 ;
- 2 orbits of size 7 in the neighborhood $\Gamma(x_0)$ (because $k = 14$);
- 12 orbits of size 7 in the second subconstituent $\Gamma_2(x_0)$ (84 vertices).

This decomposition matches the orbit structure of Cesarz and Woldar [3]. The canonical CNF encoding uses 498 primary variables: 36 internal orbit valences and 462 cross-orbit circulant variables. We encode degree, $\lambda = 1$, and $\mu = 2$ constraints into CNF and add Crawford lex-leader cuts for the 11 non-identity elements of the stabilizer $G_{\mathbb{Z}_7}$. The resulting formula has:

$$\text{Variables: } 176,613, \quad \text{Clauses: } 421,562. \quad (4.1)$$

4.2 Solver Execution and Independent DRAT Verification

We solved the canonical order-7 formula using CaDiCaL 1.9.5 [11] with binary DRAT proof logging. The solver proved the formula unsatisfiable:

```
s UNSATISFIABLE
c total process time since initialization:      771.59      seconds
c total real time since initialization:        771.76      seconds
c maximum resident set size of process:       219.38      MB
c DRAT 1098900 added clauses 47.77%
c DRAT 1201268 deleted clauses 52.23%
c DRAT 192218491 bytes (183.31 MB)
```

To verify the refutation independently of CaDiCaL, we checked the 192.2 MB (183.31 MiB) binary proof with drat-trim [12] in backward checking mode:

```
c parsing input formula with 176613 variables and 421562 clauses
c finished parsing, read 192218491 bytes from proof file
c detected empty clause; start verification via backward checking
c 213600 of 421562 clauses in core
c 495181 of 1098901 lemmas in core using 74443066 resolution steps
c 0 RAT lemmas in core; 285673 redundant literals in core lemmas
s VERIFIED
c verification time: 844.008 seconds
```

Table 1: Benchmark comparison for the order-7 automorphism of Conway’s 99-graph.

Methodology	Formulation	Solver / Tool	Runtime	Epistemic Verdict
Thakkar (2026) [8]	CP Model	OR-Tools CP-SAT	> 48 h (Timeout)	Inconclusive (UNKNOWN)
This work (Search)	Canonical CNF	CaDiCaL 1.9.5	771.59 s	s UNSATISFIABLE
This work (Certification)	Binary DRAT	drat-trim	844.01 s	PROVED (s VERIFIED)

Table 1 summarizes these results. The verified certificate closes the CP-SAT timeout case. Under our taxonomy, the formula refutation is PROVED; the translation pipeline from graph theory to CNF is COMPILED.

5 Incidence Geometry and Search Limits for Open Branches

After excluding order 7 and auditing the $f = 3$ refutations, only two prime-order actions remain open in the literature:

1. An involution ($p = 2$) with exactly $f = 1$ fixed point (Makhnev–Minakova);
2. An order-3 automorphism ($p = 3$) that is fixed-point-free (Crnković–Maksimović).

5.1 Incidence Geometry of the One-Fixed-Vertex Involution ($f = 1$)

Let $t \in \text{Aut}(G)$ be an involution with $\text{Fix}(t) = \{x_0\}$. The rooted neighborhood $\Gamma(x_0)$ consists of 7 matched pairs under t . The second subconstituent $\Gamma_2(x_0)$ contains 84 vertices partitioned into 42 transposed pairs (orbits $O_0, \dots, O_{41}$).

Each orbit $O_j = \{u, t(u)\}$ has a support in $\Gamma(x_0)$. This support is the set of matched pairs in $\Gamma(x_0)$ containing neighbors of u . We define the binary support-incidence matrix $C \in \{0, 1\}^{7 \times 42}$, where $C_{i,j} = 1$ if orbit O_j connects to the i -th matched pair of $\Gamma(x_0)$. By construction, $C = [C_1 \mid C_1]$, where $C_1 \in \{0, 1\}^{7 \times 21}$ is the vertex-edge incidence matrix of K_7 . Computing the product gives the matrix equation:

$$CC^T = 10I_7 + 2J_7. \quad (5.1)$$

Proposition 5.1 (Exact Column Weight Distribution). *For every binary matrix $C \in \{0, 1\}^{7 \times 42}$ satisfying $CC^T = 10I_7 + 2J_7$, each column has weight exactly 2.*

Proof. Let $k_j = \sum_{i=1}^7 C_{i,j}$ be the sum of column j . The total number of ones in C is $\sum_{j=1}^{42} k_j = \text{Tr}(CC^T) = \text{Tr}(10I_7 + 2J_7) = 70 + 14 = 84$. The sum of squares of column sums equals the quadratic form:

$$\sum_{j=1}^{42} k_j^2 = \mathbf{1}_7^T (CC^T) \mathbf{1}_7 = \mathbf{1}_7^T (10I_7 + 2J_7) \mathbf{1}_7 = 10(7) + 2(7^2) = 70 + 98 = 168. \quad (5.2)$$

The variance of k_j around 2 is:

$$\sum_{j=1}^{42} (k_j - 2)^2 = \sum_{j=1}^{42} k_j^2 - 4 \sum_{j=1}^{42} k_j + 4(42) = 168 - 4(84) + 168 = 168 - 336 + 168 = 0. \quad (5.3)$$

Because $(k_j - 2)^2 \geq 0$ for all j , we have $k_j = 2$ for all $j \in \{1, \dots, 42\}$. $\square$

Note that $\text{Tr}(C^T C) = \text{Tr}(CC^T) = 84$, while the quadratic form $\mathbf{1}_7^T (CC^T) \mathbf{1}_7 = 168$ gives the sum of squared column weights.

5.2 Stabilizer Decomposition and Branching Classes

The binary matrix C allows independent swaps of its 21 duplicate-column pairs. This generates a relabeling group $(S_2)^{21} \rtimes S_7$ of order 10,569,646,080. However, in the signed rooted coordinate system of the graph, the valid coordinate symmetry group is:

$$W = (\mathbb{Z}_2)^7 \rtimes S_7, \quad |W| = 2^7 \cdot 7! = 645,120. \quad (5.4)$$

Fixing a distinguished orbit O_0 with support $\{0, 1\}$, its stabilizer $\text{Stab}_W(O_0)$ has order 15,360. Under this stabilizer, the remaining 41 outer orbits partition into three canonical symmetry classes:

- **Class A (Opposite Phase, Identical Support):** Size 1, representative O_{21} (support $\{0, 1\}$, opposite sign phase).
- **Class B (Intersecting Support):** Size 20, representative O_1 with support $\{1, 6\}$ ($|\{0, 1\} \cap \{1, 6\}| = 1$).
- **Class C (Disjoint Support):** Size 20, representative O_{11} with support $\{2, 3\}$ ($|\{0, 1\} \cap \{2, 3\}| = \emptyset$).

The canonical branch representatives are $A = 21, B = 1, C = 11$. Orbit O_{10} has support $\{1, 6\}$ and belongs to Class B. Orbit O_{11} has disjoint support $\{2, 3\}$, ensuring that Class C remains disjoint from the intersecting support orbits. Unit tests in `test_z2_f1_input_audit.py` verify this property.

5.3 Limits of Combinatorial Search: The Distributed Solver Campaign

To test whether CDCL solvers can resolve the open branches without further theoretical reductions, we ran a multi-threaded search campaign using CaDiCaL on Google Cloud Platform (16 cores, dedicated instance). The search targeted the Z_2 ($f = 1$) branches and the Z_3 FPF encoding.

The campaign ran for 12.5 days and ended on September 23, 2026:

- The solvers accumulated over **2.4 billion conflicts** ($> 2.4 \times 10^9$) across multiple random seeds and variable selection heuristics.

- The solvers found neither a satisfying assignment (graph witness) nor an unsatisfiable termination (refutation).

Under our taxonomy, this campaign is classified as EXPLORED. Conflict counts, clause counts, and proof sizes do not measure the proportion of the mathematical search space eliminated. In combinatorial problems of this scale, CDCL solvers often generate redundant learned clauses without resolving the algebraic constraints. Unguided SAT solving on the $f = 1$ and Z_3 FPF branches appears intractable without additional symmetry cuts or algebraic invariants.

6 Conclusion

This report establishes a verified baseline for research on Conway’s 99-graph problem:

1. We verified foundational structural invariants and arithmetic refutations in Lean 4, distinguishing kernel-checked arithmetic from unverified graph-action hypotheses.
2. We implemented canonical SAT compilers with Crawford symmetry breaking, resolved literal-constant collision defects, and verified the pipeline with a 39-test regression suite.
3. We resolved the order-7 computational challenge. CaDiCaL produced a 192.2 MB (183.31 MiB) binary proof in 771.59 s, and drat-trim certified this proof in 844.01 s.
4. We determined the incidence geometry of the $f = 1$ involution case ($CC^T = 10I + 2J$, column weights strictly 2) and classified the multi-threaded search campaign (>2.4 billion conflicts) as EXPLORED.

The Conway 99-Graph and the Rigidity Conjecture. A central question is whether any Conway 99-graph must be rigid:

Conjecture 6.1 (Rigidity Conjecture). *If a strongly regular graph with parameters $(99, 14, 1, 2)$ exists, then $\text{Aut}(G) = \{1\}$.*

Symmetry-restricted searches cannot find a rigid graph, because they assume a non-trivial automorphism (Z_2, Z_3, Z_7) . Even if future work refutes the open $f = 1$ and Z_3 FPF branches—proving the Rigidity Conjecture—the existence of Conway’s 99-graph remains open. Progress on the general problem will require methods such as algebraic geometry or higher-order semidefinite programming (SDP) relaxations.

Declarations

Statement of AI Use. The author used large language models for code maintenance, automated test generation, and LaTeX formatting. All mathematical proofs, Lean 4 formalizations, SAT encodings, and experimental results were audited, checked by compilers, and verified by independent proof checkers.

Conflict of Interest. The author declares no competing financial or non-financial interests.

Data and Code Availability. All Lean 4 modules, Python SAT compilers, regression test suites, and reproduction scripts are openly available in the project repository.

Acknowledgments. The author thanks the developers of CaDiCaL, drat-trim, PySAT, and Lean 4 for their open-source tools.

References

- [1] John H. Conway. Five \$1,000 problems. Proceedings of the OEIS50–DIMACS Conference, OEIS Foundation, 2014. Updated 2017.
- [2] Andries E. Brouwer and Willem H. Haemers. *Spectra of Graphs*. Universitext. Springer, 2012. URL <https://homepages.cwi.nl/~aeb/math/ipm/>.
- [3] Patrick G. Cesarz and Andrew J. Woldar. On the automorphism group of a putative Conway 99-graph. *Algebraic Combinatorics*, 8(2):379–398, 2025. doi: 10.5802/alco.418.
- [4] A. A. Makhnev and I. M. Minakova. On automorphisms of strongly regular graphs with the parameters $\lambda = 1$ and $\mu = 2$. *Discrete Mathematics and Applications*, 14(2):201–210, 2004. doi: 10.1515/156939204872374.

-
- [5] A. A. Makhnev. Symmetric graphs and their automorphisms. Lecture slides, Berlin, September 2009, 2009. URL <https://www.math.uni-bielefeld.de/~baumeist/sommerschule/makhnev.pdf>. Theorem 1 attributes the prime-order fixed-point classification to Makhnev and Minakova.
 - [6] Majid Behbahani and Clement W. H. Lam. Strongly regular graphs with non-trivial automorphisms. *Discrete Mathematics*, 311(2–3):132–144, 2011. doi: 10.1016/j.disc.2010.10.005.
 - [7] Dean Crnković and Marija Maksimović. Construction of strongly regular graphs having an automorphism group of composite order. *Contributions to Discrete Mathematics*, 15(1):22–41, 2020. doi: 10.55016/ojs/cdm.v15i1.62323.
 - [8] Aalok Thakkar. A forced-structure reduction and verifiable bounds for Conway’s 99-graph. *arXiv preprint arXiv:2608.11211*, 2026.
 - [9] James M. Crawford, Matthew L. Ginsberg, Eugene M. Luks, and Amitabha Roy. Symmetry-breaking predicates for search problems. In *Proceedings of the 5th International Conference on Principles of Knowledge Representation and Reasoning (KR’96)*, pages 148–159. Morgan Kaufmann, 1996.
 - [10] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
 - [11] Armin Biere. CaDiCaL at the SAT race 2019. In *SAT Race 2019*, volume B-2019-1 of *Department of Computer Science Series of Publications B*, pages 8–9. University of Helsinki, 2019.
 - [12] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt, Jr. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, Lecture Notes in Computer Science. Springer, 2014. doi: 10.1007/978-3-319-09284-3_31.